EECE2311 - <Spring> <2026>

Lab for EECE2310: Introduction to Digital Design and Computer Architecture

Lab Assignment <02>

EECE2311 Lab Section <03>

EECE2310 Course Section <33858>

Student Name(Sijing Deng)

Date Submitted:

<Jan 30 2026>

Abstract

This lab involved designing and implementing a 4-bit adder and a 4-bit adder/subtractor using Quartus schematic capture and the DE1-SoC FPGA board. The circuits supported signed and unsigned arithmetic, included overflow detection logic derived from truth tables and Karnaugh maps, and were validated through multiple hardware test cases with results verified in decimal form. The lab strengthened understanding of two's-complement arithmetic, ripple-carry adders, and practical FPGA-based digital system testing.

Introduction

This lab is designing a circuit capable of adding and subtracting two 4-bit binary numbers.

The circuit will have an overflow indicator for both signed and unsigned numbers addition. Use the Quartus Prime Schematic to implement it. Upload the design into the FPGA

on the DE1-SoC Board and verify the design using Switches and LEDs on the board.

Lab Setup

Pre-Lab Question:

What is the difference between overflow in adding two signed binary numbers vs adding two unsigned binary numbers? Give numerical examples to support your answer.

Unsigned addition overflow occurs when the sum exceeds the maximum value that can be represented with the given number of bits. In a 4-bit unsigned system, values range from 0 to 15, so overflow happens when there is a carry out of the most significant bit (MSB).
Example:
1111_2(15) + 0001_2(1) = $1\ 0000_2$.
The 4-bit result is $0000_2$with a carry out, so overflow occurred.

Signed addition overflow (using two's complement) happens when adding two numbers with the same sign produces a result with the opposite sign. In 4-bit two's-complement, the range is −8 to +7, and overflow is detected by comparing the carry into and out of the MSB or by checking sign bits.
Example 1 (positive + positive overflow):
0100_2(+4) + 0101_2(+5) = 1001_2, which is −7 in two's complement—an overflow because two positives produced a negative result.

Example 2 (negative + negative no overflow):
1011_2(−5) + 1101_2(−3) = $1\ 1000_2$.
The 4-bit result 1000_2represents −8, but the true sum is −8? Actually −5 + −3 = −8, which fits
Example 3 (negative + negative overflow):
1010_2(−6) + 1011_2(−5) = 0101_2(+5), which is overflow because two negatives produced a positive result.

Equipment

- DE1-SoC FPGA development board
- Computer with Intel Quartus Prime software installed
- USB programming cable for FPGA configuration
- Previously designed half-adder and full-adder schematic files
- Connecting cables and power supply for the board

4-bit Adder / Subtractor							
Inputs				Outputs			
Cases	Carry-in	A	B	A + B	Carry - out	A - B	Borrow out
small numbers	0	0001	0101	0110	0	1100	1
small numbers	1	0001	0101	0111	0	1011	1
larger + smaller	0	1001	0001	1010	0	1000	0
larger + 0	1	1001	0000	1010	0	1000	0
mid-range	0	0110	0010	1000	0	1000	0
mid-range	1	0110	0010	1001	0	0100	0
subtraction gives negative	0	0001	0111	1000	0	0011	1
subtraction gives negative	1	0001	0111	1001	0	1010	1
max value (15), overflow	0	1111	1111	1110	0	0000	0
max value (15), overflow	1	1111	1111	1111	1	1111	1

Table 1. Test Bench for 4 -bit Adder / Subtractor

Results and Analysis of Four Bit Adder

A3	B3	S3	Overflow
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

Table 2. Overflow Detection Truth Table

• A3 = MSB of A
• B3 = MSB of B
• S3 = MSB of Sum

Four Bit Adder Schematic

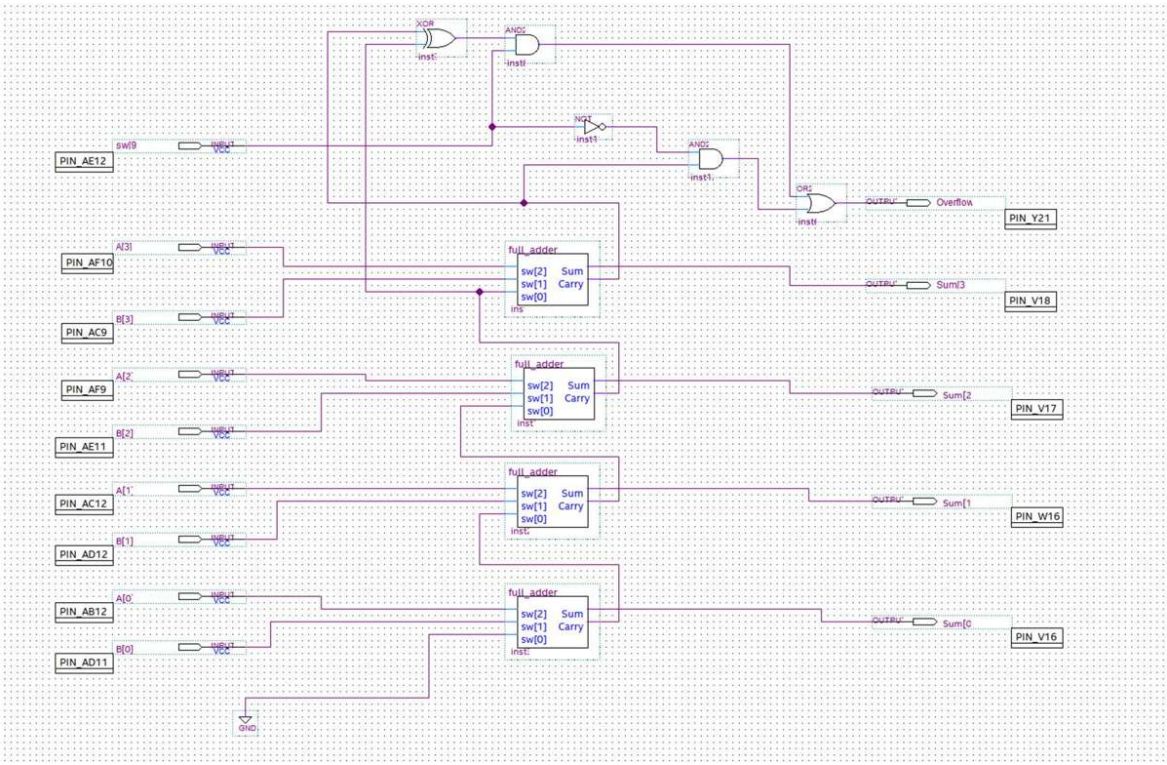


Figure 1. 4 - Bit Adder Schematic

Four Bit Adder Schematic Switches Assignments

Signed/Unsigned(1,0)	B[3]	B[2]	B[1]	B[0]	A[3]	A[2]	A[1]	A[0]
sw[9]	sw[7]	sw[6]	sw[5]	sw[4]	sw[3]	sw[2]	sw[1]	sw[0]

Table 3. Switches Assignments

Note for Four Bit Adder Schematic

1. Inputs and Outputs

- **Inputs:**

  A[3-0]and B[3-0] come from board switches and represent the two 4-bit numbers.
  SW9 selects signed vs. unsigned mode for overflow detection.
  The least-significant carry-in is tied to GND, so the addition starts with 0.

- **Outputs:**

  Sum[3:0] go to LEDs and represent the 4-bit result.

  Overflow goes to LED9 and lights when overflow occurs.

2. Ripple-Carry Adder Structure

- **Bit 0 (LSB):**
  Inputs: A0, B0, Carry-in = 0
  Outputs: Sum0 and Carry-out to next stage.
- **Bit 1:**
  Takes Carry-in from Bit 0.
  Outputs Sum1 and next carry.
- **Bit 2:**
  Takes Carry-in from Bit 1.
  Outputs Sum2.
- **Bit 3 (MSB):**
  Takes Carry-in from Bit 2.
  Outputs Sum3 and final carry.

3. Overflow Detection Logic

Unsigned Overflow

when the carry-out of the MSB = 1.

Signed Overflow (Two's Complement)

The carry into the MSB ≠ carry out of the MSB

OR when two numbers with the same sign produce a result with a different sign.

4. SW[9] – Signed/Unsigned

- SW9 = 0 → unsigned overflow logic active.
- SW9 = 1 → signed overflow logic active.

Results

Result 1: Unsigned 5 + 3

Unsigned (No Overflow)	
5 + 3 = 8	
sw[9]	0
A[3-0]	0101 (5)
B[3-0]	0011 (3)
Expected Sum	1000 (8)
Overflow	0

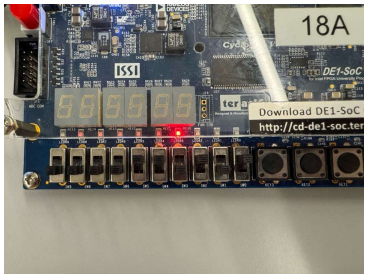


Figure 2. A[3-0](0101), B[3-0](0011)

Result 2: Unsigned 15 + 1

Unsigned (Overflow)	
15 + 1 = 16	
sw[9]	0
A[3-0]	1111 (15)
B[3-0]	0011 (1)
Expected Sum	0000 (16)
Overflow	1

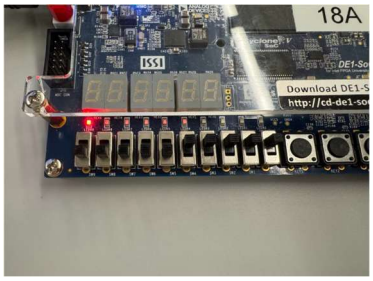


Figure 3. A[3-0](1111), B[3-0](0001)

Result 3: Signed 3 + 2

Signed (Positive + Positive) (No Overflow)	
3 + 2 = 5	
sw[9]	1
A[3-0]	0011 (3)
B[3-0]	0010 (2)
Expected Sum	0101 (5)
Overflow	0

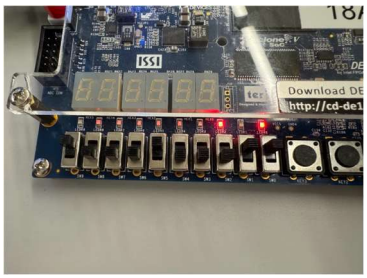


Figure 4. A[3-0](0011), B[3-0](0010)

Result 4: Signed 4 + 4

Signed (Positive + Positive) (Overflow)	
4 + 4 = 8	
sw[9]	1
A[3-0]	0100 (4)
B[3-0]	0100 (4)
Expected Sum	1000 (8)
Overflow	1

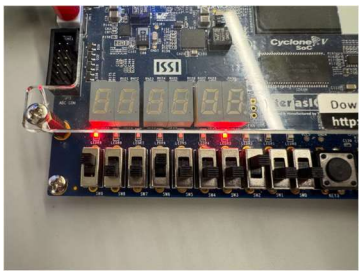


Figure 5. A[3-0](0100), B[3-0](0100)

Result 5: Signed -2 + -1

Signed (Negative + Negative) (No Overflow)	
-2 + -1 = -3	
sw[9]	1
A[3-0]	1110 (-2)
B[3-0]	1111 (-1)
Expected Sum	1101 (-3)
Overflow	0

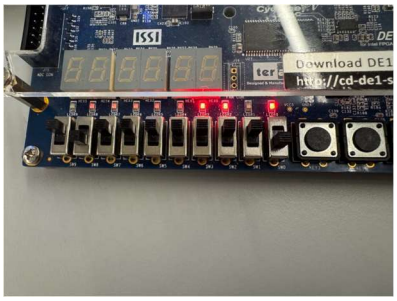


Figure 6. A[3-0](1110), B[3-0](1111)

Result 6: Signed -5 + -4

Signed (Negative + Negative) (Overflow)	
-5 + -4 = -9	
sw[9]	1
A[3-0]	1011 (-5)
B[3-0]	1100 (-4)
Expected Sum	0111 (-9)
Overflow	1

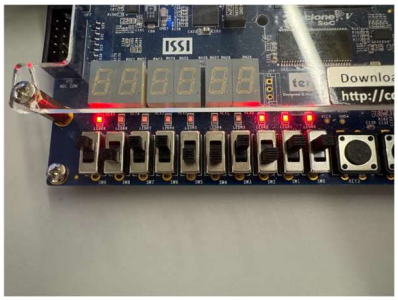


Figure 7. A[3-0](1011), B[3-0](1100)

Result 7: Signed 5 + -3

Signed (Positive + Negative) (No Overflow)	
5 + -3 = 2	
sw[9]	1
A[3-0]	0101 (5)
B[3-0]	1101 (-3)
Expected Sum	0010 (2)
Overflow	0

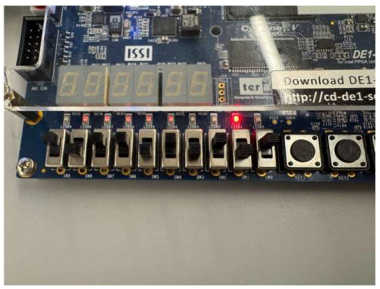


Figure 8. A[3-0](0101), B[3-0](1101)

Result 8: Signed -4 + 3

Signed (Negative + Positive) (No Overflow)	
-4 + 3 = -1	
sw[9]	1
A[3-0]	1100 (-4)
B[3-0]	0011 (3)
Expected Sum	1111 (-1)
Overflow	0

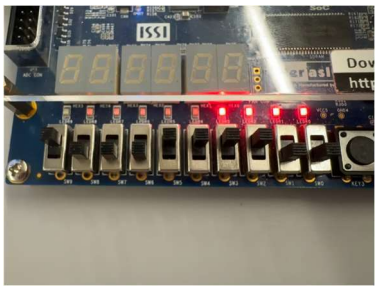


Figure 9. A[3-0](1100), B[3-0](0011)

Results and Analysis of Four Bit Adder/Subtractor

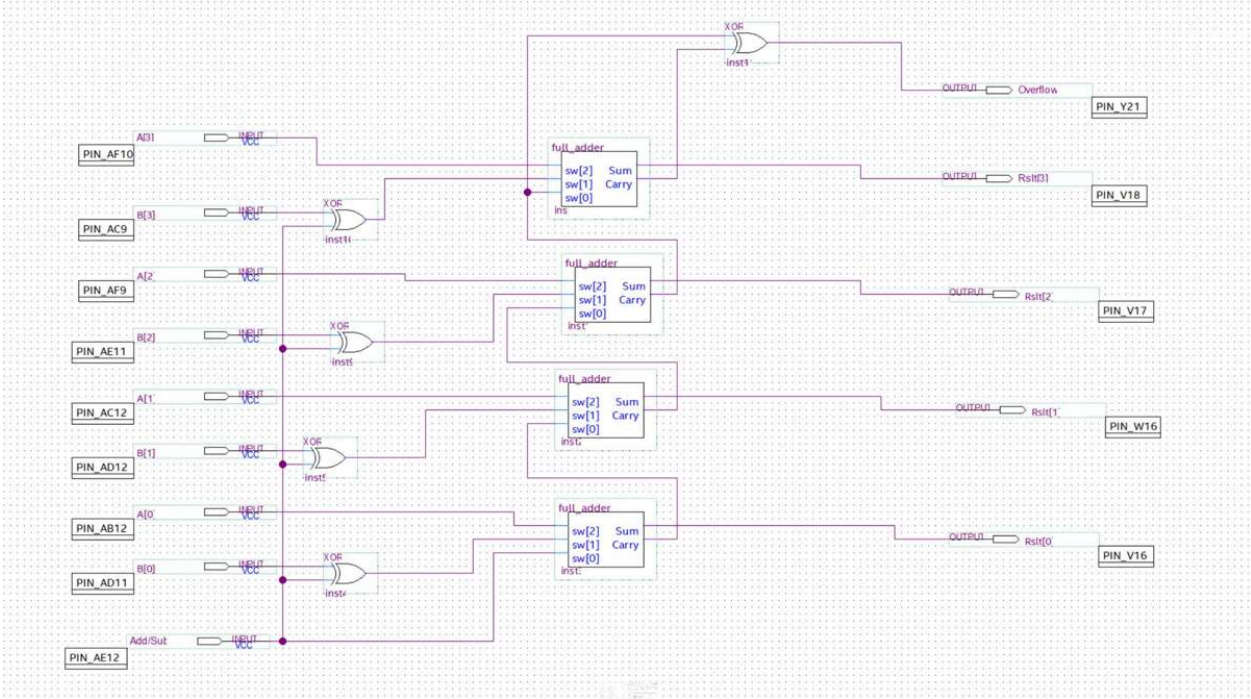


Figure 10. 4 - Bit Adder/Subtractor Schematic

Add/Subtract(0,1)	B[3]	B[2]	B[1]	B[0]	A[3]	A[2]	A[1]	A[0]
sw[9]	sw[7]	sw[6]	sw[5]	sw[4]	sw[3]	sw[2]	sw[1]	sw[0]

Table 4. Add/Subtractor Switches Assignment

Note for Four Bit Subtractor Adder/Subtractor Schematic

1. Inputs and Outputs

Inputs:

- A[3:0] and B[3:0] come from board switches and represent the two 4-bit operands.
- Add/Sub is a control signal:

  0 → Addition (A + B)

  1 → Subtraction (A − B)
- The Add/Sub signal is also connected to the least-significant carry-in.

Outputs:

- Result[3:0] go to LEDs and display the 4-bit sum or difference.
- Overflow goes to an LED and indicates signed overflow.

2. Subtraction Using Two's Complement

$$A - B = A + (\sim B) + 1$$

- Each B[i] is XORed with Add/Sub.

  If Add/Sub = 0, B passes through unchanged.

  If Add/Sub = 1, B is inverted.
- Add/Sub is also used as the initial carry-in, adding the required +1.

3. Ripple-Carry Structure

- **Bit 0 (LSB):**
  Inputs: A0, (B0 ⊕ Add/Sub), Carry-in = Add/Sub
  Outputs: Result0 and Carry-out to Bit 1
- **Bit 1:**
  Inputs: A1, (B1 ⊕ Add/Sub), Carry-in from Bit 0
  Outputs: Result1 and Carry-out
- **Bit 2:**
  Inputs: A2, (B2 ⊕ Add/Sub), Carry-in from Bit 1
  Outputs: Result2 and Carry-out
- **Bit 3 (MSB):**
  Inputs: A3, (B3 ⊕ Add/Sub), Carry-in from Bit 2
  Outputs: Result3 and final carry

4. Overflow Detection Logic

The carry into the MSB ≠ carry out of the MSB
If they differ, Overflow = 1
If they are the same, Overflow = 0

#	Operation	A (dec)	B (dec)	Switches 4 (A)	1– Switches 8 (B)	5– Switch 9	Expected Rslt (dec)	Rslt LEDs	Overflow (LED 9)
1	Add	+3	+2	0011	0010	0	+5	0101	0
2	Add	+7	+2	0111	0010	0	−7 (overflow)	1001	1
3	Add	−3	−2	1101	1110	0	−5	1011	0
4	Add	−7	−2	1001	1110	0	+7 (overflow)	0111	1
5	Add	+3	−2	0011	1110	0	+1	0001	0
6	Add	−3	+2	1101	0010	0	−1	1111	0
7	Subtract	+3	+2	0011	0010	1	+1	0001	0
8	Subtract	+2	+3	0010	0011	1	−1	1111	0
9	Subtract	−3	−2	1101	1110	1	−1	1111	0
10	Subtract	−2	−3	1110	1101	1	+1	0001	0
11	Subtract	+7	−2	0111	1110	1	+9 → −7 (overflow)	1001	1
12	Subtract	−7	+2	1001	0010	1	−9 → +7 (overflow)	0111	1

Table 5. 12 Test Cases 4-Bit Adder/Subtractor

Result 2: Add 3 + 2

Add (No Overflow) (Positive + Positive = Positive)	
3 + 2 = 5	
sw[9]	Add(0)_
A[3-0]	0011 (3)
B[3-0]	0010 (2)
Expected Rslt	0101 (5)
Overflow	0

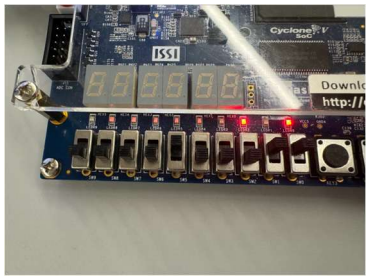


Figure 11. A[3-0](0011), B[3-0](0010)

Result 2: Add 7 + 2

Add (Overflow) (Positive + Positive = Positive)	
7 + 2 = 9	
sw[9]	Subtract(1)
A[3-0]	0111 (7)
B[3-0]	0010 (2)
Expected Rslt	1001 (-7)
Overflow	1

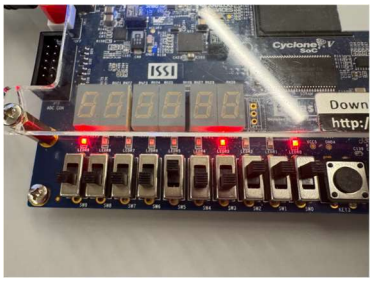


Figure 12. A[3-0](0111), B[3-0](0010)

Result 3: Add -3 + -2

Add (No Overflow) (Negative + Negative = Negative)	
-3 + -2 = -5	
sw[9]	Add(0)
A[3-0]	1101 (-3)
B[3-0]	1110 (-2)
Expected Rslt	1011 (-5)
Overflow	0

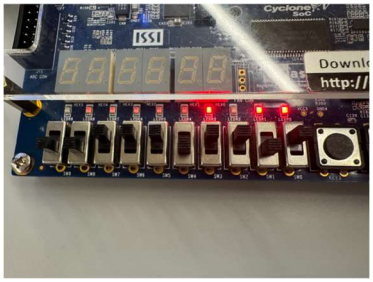


Figure 13. A[3-0](1101), B[3-0](1110)

Result 4: Add -7 + -2

Add (Overflow) (Negative + Negative = Negative)	
-7 + -2 = -9	
sw[9]	Subtract(1)
A[3-0]	1001 (-7)
B[3-0]	1110 (-2)
Expected Rslt	0111 (7)
Overflow	1

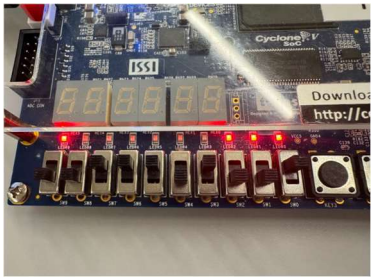


Figure 14. A[3-0](1001), B[3-0](1110)

Result 5: Add 3 + -2

Add (No Overflow) (Positive + Negative = Positive)	
3 + -2 = 1	
sw[9]	Add(0)
A[3-0]	0011(3)
B[3-0]	1110 (-2)
Expected Rslt	0001 (1)
Overflow	0

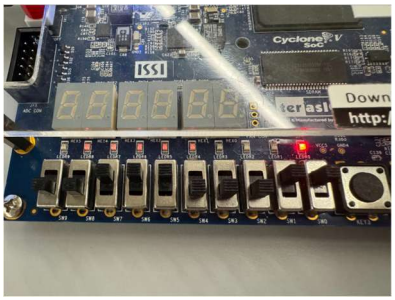


Figure 15. A[3-0](0011), B[3-0](1110)

Result 6: Add -3 + 2

Add (No Overflow) (Negative + Positive = Negative)	
-3 + 2 = -1	
sw[9]	Add(0)
A[3-0]	1101 (-3)
B[3-0]	0010 (2)
Expected Rslt	1111 (-1)
Overflow	0

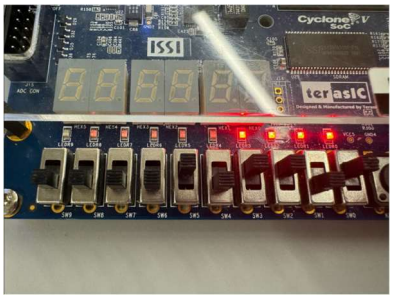


Figure 16. A[3-0](1101), B[3-0](0010)

Result 7: Subtract 3 - 2

Subtract (No Overflow) (Positive – Positive = Positive)	
3 - 2= 1	
sw[9]	Subtract(1)
A[3-0]	0011 (3)
B[3-0]	0010 (2)
Expected Rslt	0001 (1)
Overflow	0

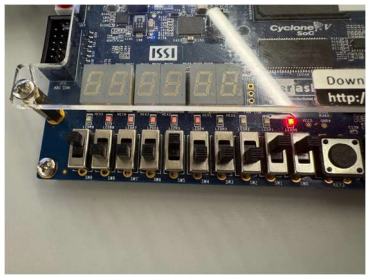


Figure 17. A[3-0](0011), B[3-0](0010)

Result 8: Subtract 2 - 3

Subtract (No Overflow) (Positive – Positive = Negative)	
2 - 3 = -1	
sw[9]	Subtract(1)
A[3-0]	0010 (2)
B[3-0]	0011 (3)
Expected Rslt	1111 (-1)
Overflow	0

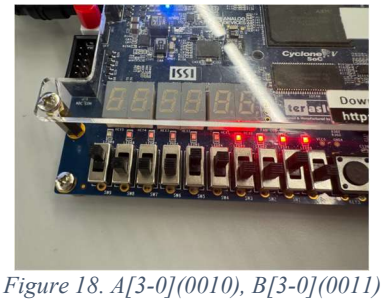


Figure 18. A[3-0](0010), B[3-0](0011)

Result 9: Subtract -3 – (- 2)

Subtract (No Overflow) (Negative – Negative = Negative)	
-3 – (-2) = -1	
sw[9]	Subtract(1)
A[3-0]	1101 (-3)
B[3-0]	1110 (-2)
Expected Rslt	1111 (-1)
Overflow	0

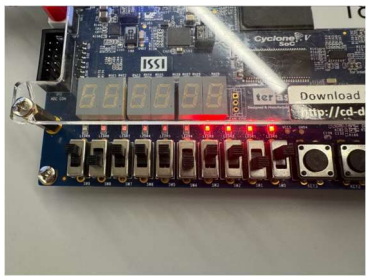


Figure 19. A[3-0](1101), B[3-0](1110)

Result 10: Subtract -2 – (-3)

Subtract (No Overflow) (Negative – Negative = Positive)	
-2 – (-3) = 1	
sw[9]	Subtract(1)
A[3-0]	1110 (-2)
B[3-0]	1101 (-3)
Expected Rslt	0001 (1)
Overflow	0

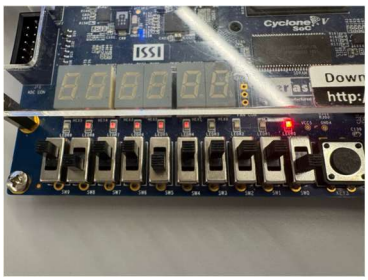


Figure 20. A[3-0](1110), B[3-0](1101)

Result 11: Subtract 7 – (-2)

Subtract (Overflow) (Positive – Negative = Positive)	
7 – (-2) = 9	
sw[9]	Subtract(1)
A[3-0]	0111 (7)
B[3-0]	1110 (-2)
Expected Rslt	1001 (-7)
Overflow	1

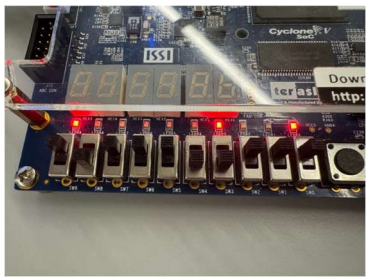


Figure 21. A[3-0](0111), B[3-0](1110)

Result 12: Subtract -7 - 2

Subtract (Overflow) (Negative – Positive = Negative)	
-7 - 2 = -9	
sw[9]	Subtract(1)
A[3-0]	1001 (-7)
B[3-0]	0010 (2)
Expected Rslt	0111 (7)
Overflow	1

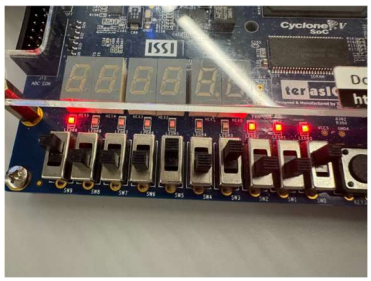


Figure 22. A[3-0](1001), B[3-0](0010)

Conclusion

Note for the Add/Subtractor Result

This result showed correct operation of the 4-bit adder–subtractor using two's complement arithmetic. Switch SW9 selects addition or subtraction, and the output LEDs display the signed result. The overflow LED correctly indicates signed overflow when the computed result exceeds the representable range, confirming proper arithmetic and overflow detection.

Analysis

All results matched theoretical expectations for signed two's complement arithmetic, and no further analysis was necessary.

At a system level, these results demonstrate that a single adder/subtractor unit can efficiently perform multiple arithmetic operations using minimal additional hardware. By reusing the same full-adder chain for both addition and subtraction, the design illustrates a fundamental arithmetic logic unit (ALU) concept used in processors. Although ripple-carry adders are not optimal for high-speed applications due to carry propagation delay, they are well-suited for small bit-width designs.

References

[1] Michael Benjamin, "*Lab Report Guide*", Northeastern University, January 18, 2006.

[2] EECE 2311 lab02 pdf.